College of IT

Department of Computer Science

ITCS440 Intelligent Systems

Course Project

Chosen topic

Path Finding Agent

Objective

Implement an AI agent that finds the shortest path between two points in a grid or maze using BFS, DFS, and A* algorithms.

Brief Description

This project's goal is to create and execute a smart pathfinding agent capable of traversing a grid and determining the shortest route from a starting point to a goal point. The environment is modeled as a 2D matrix, and three traditional AI search algorithms: BFS, DFS, and A*are used to evaluate their performance by time taken, number of visited nodes, and final path length. A GUI is used to effectively show each algorithm exploring and paths and finding the final path.

Prepared by:

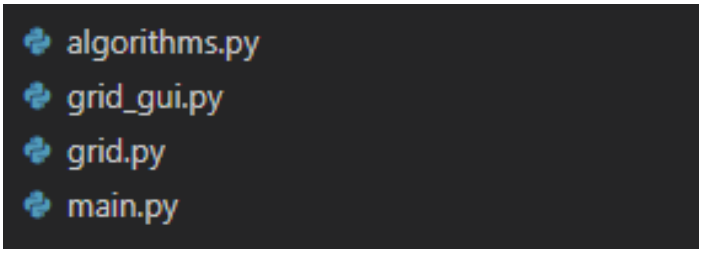
Abdelrahman Adel

Haitham Al Amri

Mohammed Mustaf

System Design and Implementation

To build the system, we divided the project into several modules that each serve a specific purpose. The grid module is responsible for constructing the 15×15 matrix and initializing all cells as free. The algorithms module contains the full implementations of BFS, DFS, and A*. Each algorithm returns both the list of visited nodes and the final path that connects the start to the goal. The visualization module draws the grid using a graphical interface and highlights visited cells and the final path using different colors. Finally, the main module connects everything together by handling user interactions, running the selected algorithm, timing the execution, and animating the search process.



Each algorithm behaves differently in practice. BFS explores the grid in layers, gradually moving outward from the starting point. Because it expands all equally distant nodes before moving deeper, it always finds the shortest path when one exists. DFS behaves more aggressively: it immediately dives deep into one direction, often reaching dead ends before backtracking. This can produce long, inefficient paths and unnecessary exploration. A* takes a more intelligent approach by using the Manhattan distance to estimate how close each node is to the goal. This heuristic guides the algorithm toward promising directions, allowing it to find optimal paths with fewer explored nodes.

BFS DFS A*

The Pygame interface makes the project interactive and easy to test. The user can place walls by clicking on cells, and pressing B, D, or A triggers BFS, DFS, or A*. The program prints the number of visited nodes, path length, and time taken, while the screen animates the search by coloring the explored cells in violet and the final path in bright blue. This visual feedback became an essential part of understanding how each algorithm behaves on different types of grids.

Results and Discussion

We tested BFS, DFS, and A* on several grid configurations, ranging from empty grids to layouts with moderate and heavy obstacles.

1 – Empty grid

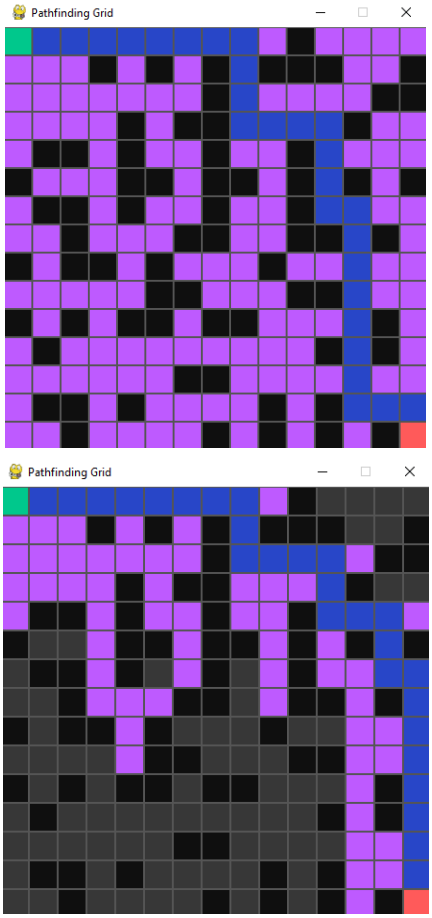
	Visited cells	Path length	Time
BFS	225	29	0.28 ms
DFS	113	113	0.17 ms
A*	225	29	0.58 ms

2 – Moderate number of walls

	Visited cells	Path length	Time
BFS	191	29	0.23 ms
DFS	116	53	0.18 ms
A*	182	29	0.47 ms

3 – Big number of walls

	Visited cells	Path length	Time
BFS	115	29	0.20 ms
DFS	118	39	0.16 ms
A*	85	29	0.23 ms



BFS and A* on a crowded grid

On empty grid and grid with low number of walls A* and BFS behaved similarly, exploring big number of cells but always finding the optimal path to the goal, while DFS explored fewer number of cells but the path to goal it found was always not optimal. When a large number of walls was added and the grid was maze-like, A* explored the smallest number of cells while still producing the optimal path thanks to Manhattan heuristic guiding the search. BFS also produced optimal path but it explored all of the grid. DFS performed the worst overall, never providing an optimal solution, it's fast but not optimal.

A* behaves very similarly to BFS on an empty grid because the Manhattan distance heuristic does not create any meaningful preference between different directions when there are no obstacles forcing the algorithm to make choices. In an open 15×15 grid, every position that is the same distance from the goal has the same heuristic value, which means many nodes end up with identical or nearly identical f-scores. Since our A* implementation pushes nodes into a priority queue sorted by f, g and the heuristic does not strongly favor one direction over another, the tie breaking naturally causes A* to expand nodes in a wide, uniform pattern. This expansion closely resembles BFS, which also spreads

outward from the start in layers. In other words, because the Manhattan heuristic perfectly aligns with the true distance on an empty grid, A* cannot eliminate any area of the search space. As a result, it explores most of the grid before reaching the goal, producing the same optimal path length as BFS and visiting almost the same number of cells. Only when walls force detours, the heuristic begin to matter, allowing A* to behave differently and more efficiently than BFS.

BFS VS A* on an empty grid (purple highlights visited cells)

Conclusion

This project gave us a clear, hands-on understanding of how different AI search algorithms behave in real environments. Implementing BFS, DFS, and A* from scratch helped us see the strengths and weaknesses of each approach, especially once we visualized their exploration patterns on the grid. BFS consistently delivered the shortest path but explored many nodes, while DFS often reached the goal with fewer explored nodes but produced very long, inefficient routes. A* proved to be the most practical option on obstacle-heavy grids, balancing search efficiency with optimal path quality through its heuristic guidance. Building the GUI also taught us how valuable visualization is for debugging and understanding algorithm behavior, and it made the differences between the algorithms much easier to interpret. Overall, the project successfully met all requirements and strengthened our understanding of search algorithms, heuristics, problem representation, and performance analysis in AI.